Anexo E —— Guía para la Creación de un Proyecto Base del Neorv32 en Vivado 2024.2.

E.1. Requisitos Previos.

E.1.1. Archivos RTL del NEORV32.

Antes de la creación del nuevo proyecto, se debe haber realizado una copia de los archivos RTL del procesador disponibles en `/neorv32/rtl/core`:

```
~/neorv32/rtl/core$ cp -r . ~/Desktop/neorv32_core_files
```

E.1.2. Test Setup.

Para configurar el procesador se empleará una de las plantillas con la configuración mínima de este. Para ello se necesita hacer una copia del archivo `neorv32_test_setup_bootloader.vhd` en el caso de realizar la carga del programa de usuario (`main.c`) vía *bootloader*:

```
~/neorv32/rtl/test_setups$ cp neorv32_test_setup_bootloader.vhd ~/Desktop
```

Si se prefiere integrar el programa de usuario directamente en la memoria IMEM del procesador se realizará una copia del archivo `neorv32_test_setup_approm.vhd`:

```
~/neorv32/rtl/test_setups$ cp neorv32_test_setup_approm.vhd ~/Desktop
```

E.1.3. Board File (Nexys4 DDR).

Para que el proyecto de Vivado se adapte a la placa empleada en este trabajo se necesita el archivo de definición de la placa (*Board File*). En el Anexo D se detalla la instalación del archivo.

E.2. Creación del Proyecto.

1. Ejecutar el programa Vivado versión 2024.2.
2. En la ventana de inicio seleccionar la opción `Create Project` (Fig.1).
3. En la ventana emergente pulsamos `Next` para continuar.
4. Indicamos el nombre del proyecto en el campo `Project name` y seleccionamos la dirección de guardado con `Project location`. Pulsamos en `Next` para continuar.
5. En la ventana `Project Type` seleccionamos solo la opción `RTL Project` y pulsamos `Next` (Fig.2).

6. En la ventana `Add Sources` se debe seleccionar la opción `VHDL` en los campos `Tarjet language` y `Simulator language`. Una vez hecho esto pulsar sobre `Add Files` (Fig.3).

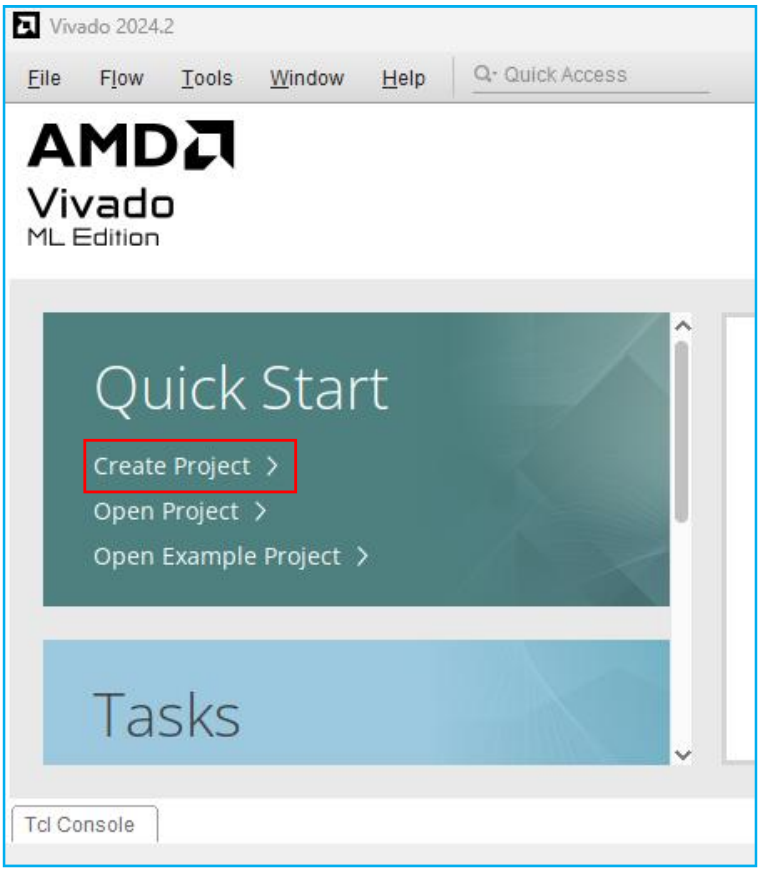

Fig.1. Ventana de inicio de Vivado, se resalta la opción `Create Project`.

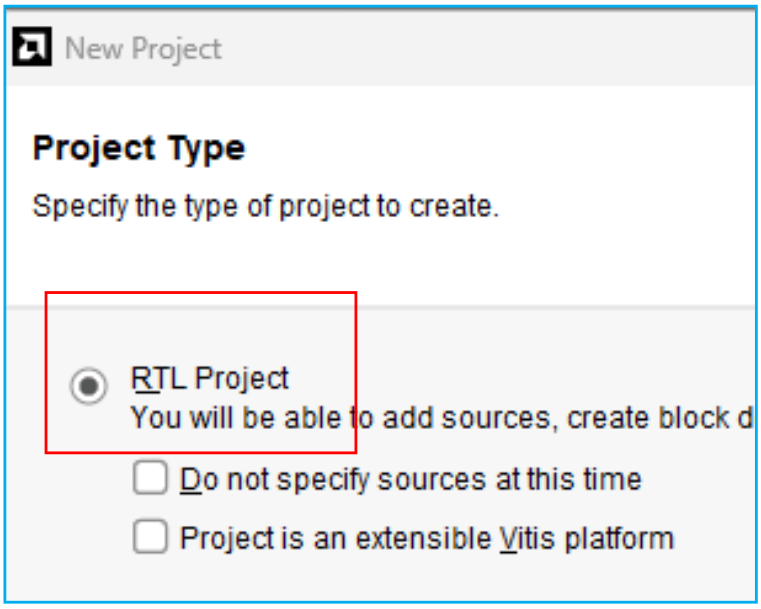

Fig.2. Ventana `Project Type`, aquí se elige el tipo de proyecto, solo debe seleccionarse el campo `RTL Project`.

7. En la ventana `Add Sources Files` acceder a la dirección donde se encuentren alojados los archivos RTL del NEORV32. Seleccionar todos los archivos y pulsar en `OK` (Fig.4).

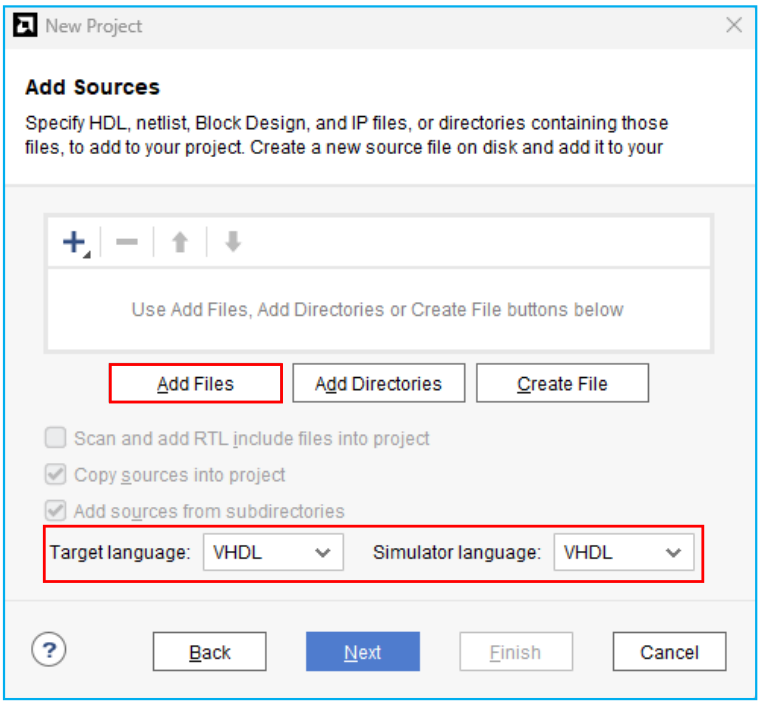

Fig.3. Ventana `Add Sources`, aquí se añaden los archivos RTL del proyecto en lenguaje de descripción hardware, VHDL en este caso.

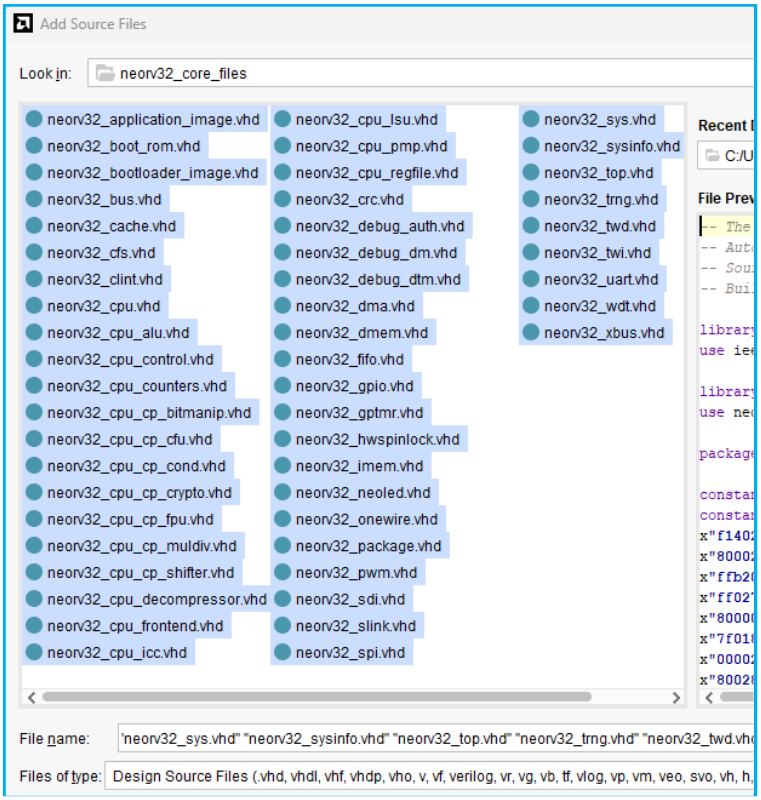

Fig.4. Ventana `Add Sources Files`.

8. Con los archivos añadidos, se debe seleccionar la opción `Copy sources into Project`, lo cual es importante a la hora de migrar el proyecto a otros dispositivos (Fig.5). Es recomendable comprobar que el número de archivos sea igual a 51.

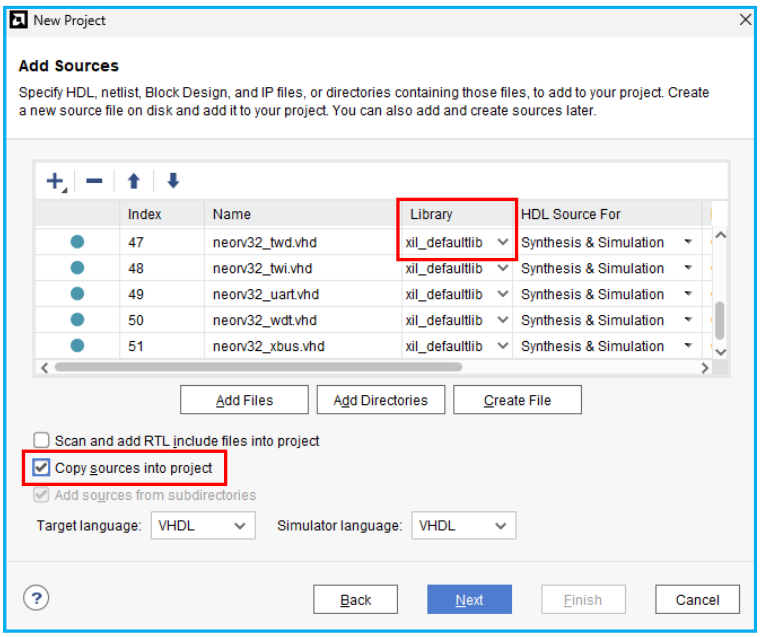

Fig.5. Archivos RTL del NEORV32 seleccionados mediante `Add Files`.

9. Todavía en la ventana `Add Sources`, si se observa el campo `Library`, la carpeta por defecto donde Vivado aloja los archivos es `xil_defaultlib` (Fig.6). La carpeta debe renombrarse como `neorv32`[1].

10. Dado que cambiar todos los campos `Library` del total de los archivos puede resultar tedioso, una forma más sencilla consiste en pulsar sobre `Add Directories`, en lugar de `Add Files`. En este caso seleccionaríamos la carpeta `neorv32_core_files` que aloja a todos los archivos anteriores (Fig.7).

11. Ahora para renombrar el campo `Library` como `neorv32` basta con hacerlo una solo vez (Fig.8). Pulsamos en `Next` para continuar. Si se hubiera olvidado cambiar la librería, se podría modificar más adelante.

12. En la ventana `Add Constraints` se debe añadir el archivo de restricción de pines de la placa Nexys4 DDR `Nexys-4-DDR-Master.xdc` (Fig.9) disponible en: https://github.com/Digilent/digilent-xdc/blob/master/Nexys-4-DDR-Master.xdc

[1] En los proyectos del NEORV32 es importante que los archivos se alojen en una carpeta llamada `neorv32`. Esto se debe a simple compatibilidad por cómo está escrita la instanciación de entidades VHDL dentro de la jerarquía del proyecto NEORV32.

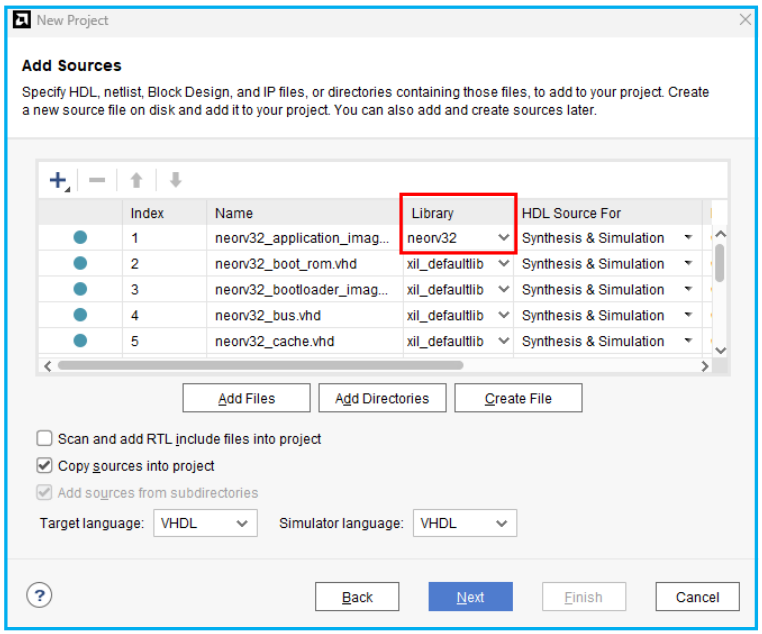

Fig.6. Cambio a `neorv32` del campo `Library` para el caso de `Add Files`, hay que cambiar todos los archivos uno a uno.

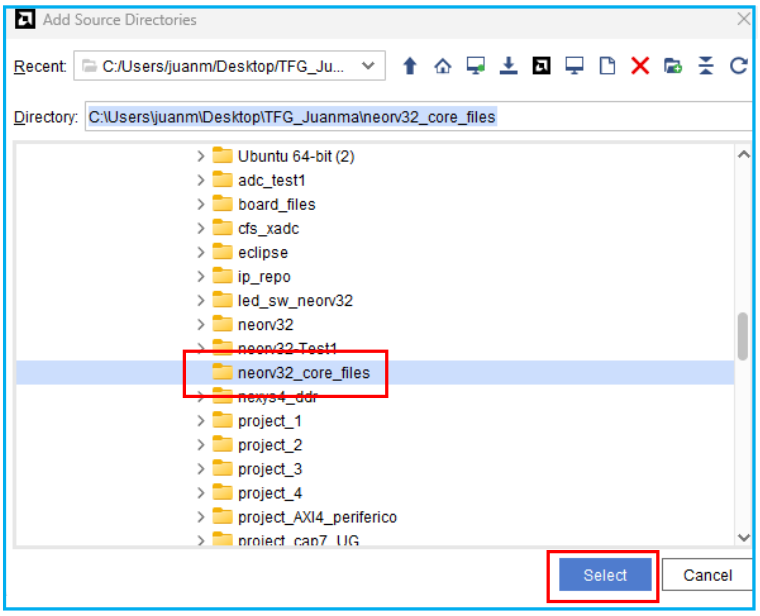

Fig.7. Ventana `Add Source Directories`, se debe seleccionar la carpeta que contenga todos los archivos RTL del NEORV32, en este caso `neorv32_core_files`.

13. Una vez añadido el archivo de restricción de pines, pulsamos en `Next` para continuar.

14. En la ventana `Default Part` escogemos la opción `Boards` y seleccionamos Nexys4 DDR (Fig.10). Para esto es necesario haber realizado el apartado 1.3 de la sección de prerrequisitos. Pulsamos en `Next`.

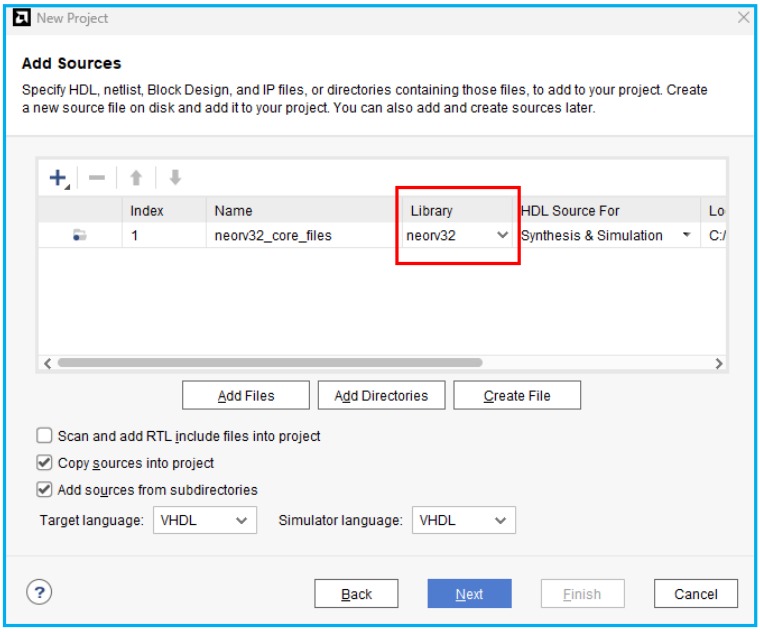

Fig.8. Cambio a `neorv32` del campo `Library` para el caso de `Add Directories`, en este caso solo es necesario cambiarlo una vez.

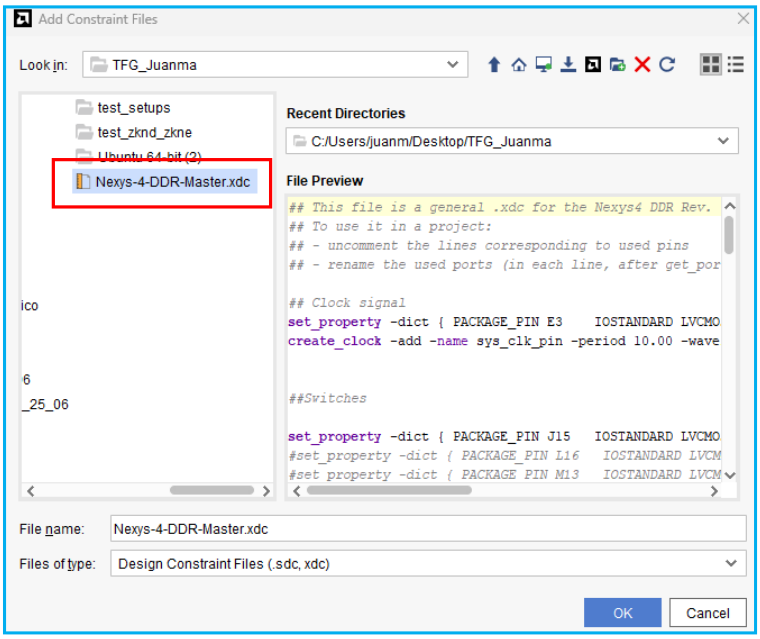

Fig.9. Ventana `Add Constraints Files`, se debe seleccionar el archivo `Nexys-4-DDR-Master.xdc`.

15. Si todo ha ido bien, la ventana de resumen de proyecto debería ser similar a la Fig.11. Una vez verificado el resumen del proyecto, pulsamos en `Finish`.

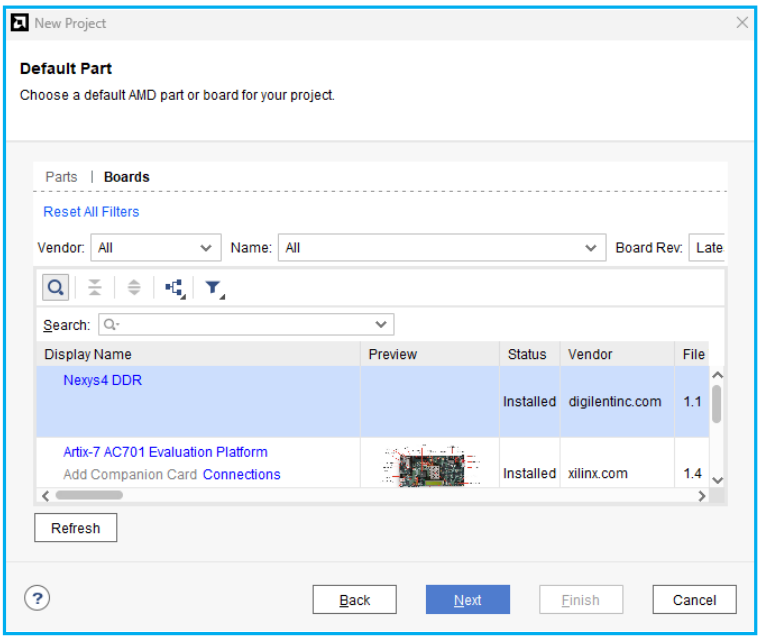

Fig.10. Ventana `Default Part`, se debe seleccionar el *board file* correspondiente a la placa Nexys4 DDR.

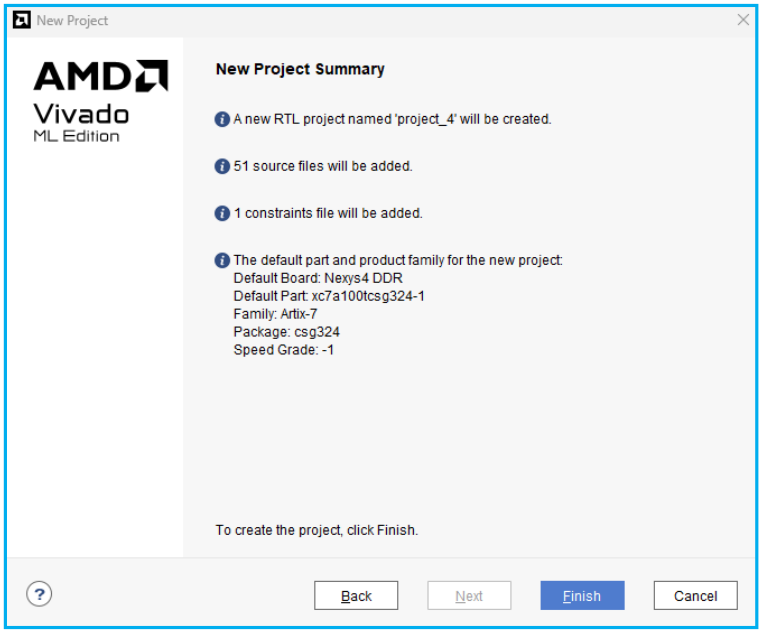

Fig.11. Ventana de resumen de la creación del proyecto.

16. En la ventana `Sources`, si se cambió correctamente el nombre por defecto del campo `Library` por neorv32, nos debe aparecer en la pestaña `Libraries` todos los archivos del procesador dentro de esta (Fig.12). De no ser así, pulsaremos en `Settings` dentro del `Project Manager` (Fig.13).

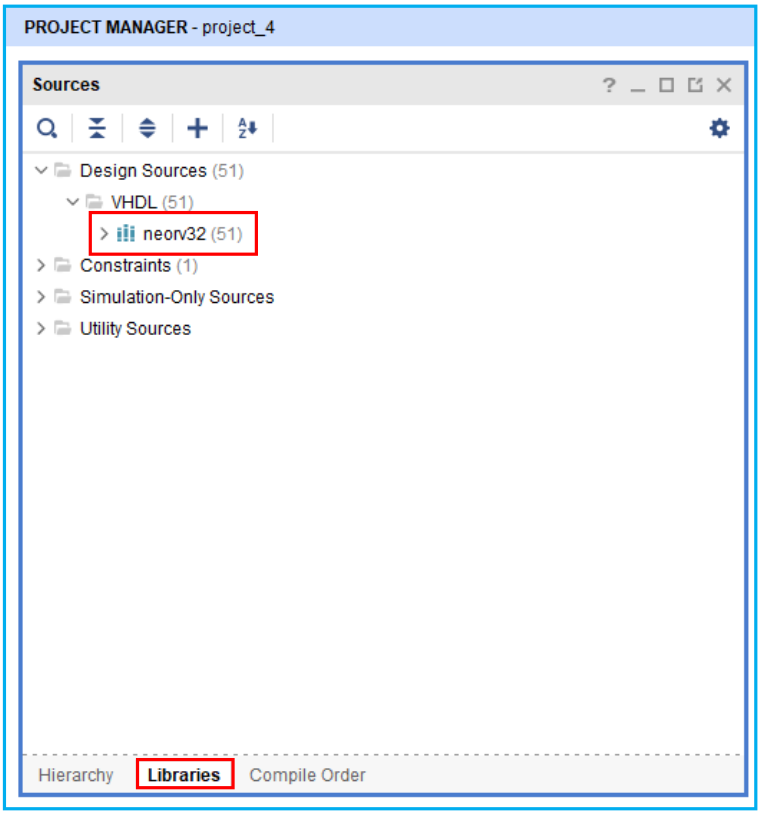

Fig.12. Ventana `Sources`, dentro de la pestaña `Libraries` se observa como todos los archivos fuente están dentro de la carpeta neorv32.

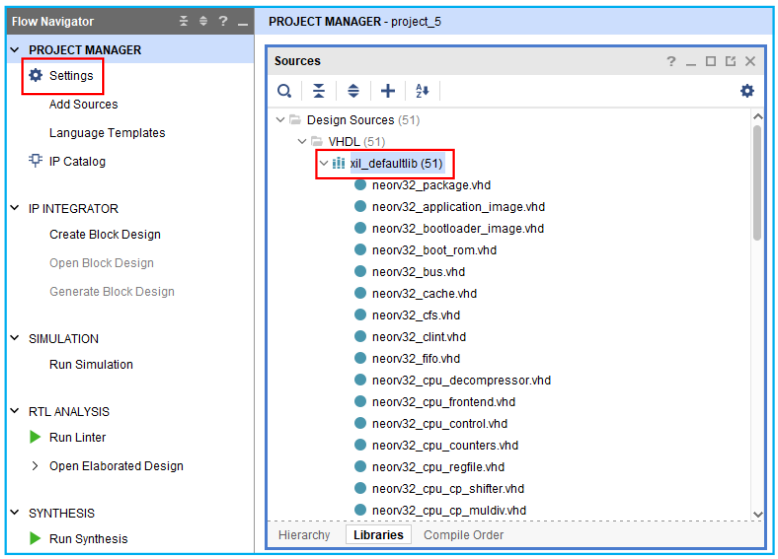

Fig.13. Pestaña `Libraries` de la ventana `Sources`, en este caso la librería de los archivos fuente no es neorv32.

17. Dentro de la ventana `Settings`, debe seleccionarse la opción `General` ubicada en `Project Settings`. Una vez hecho, se cambiará el nombre del campo `Default Library` por neorv32 (Fig.14). Pulsamos `Apply` y `OK`.

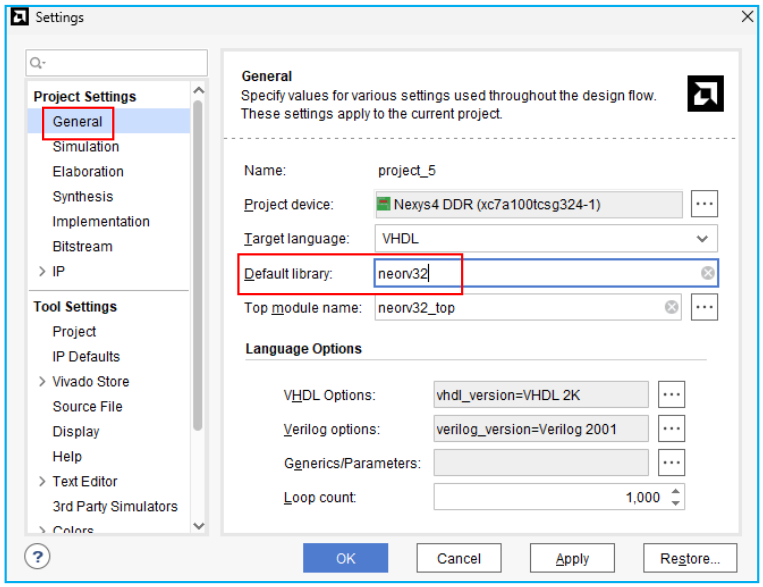

Fig.14. Ventana `Settings`, cambio de nombre de la librería por defecto.

18. El siguiente paso es añadir el archivo de jerarquía superior, neorv32_test_setup_bootloader.vhd o neorv32_test_setup_approm.vhd, según el método que se vaya a seguir para ejecutar el programa de aplicación. Para hacerlo pulsamos en `Add Sources` situado bajo `Settings`, en la ventana emergente seleccionamos `Add or create design sources` y pulsamos en `Next` (Fig.15).

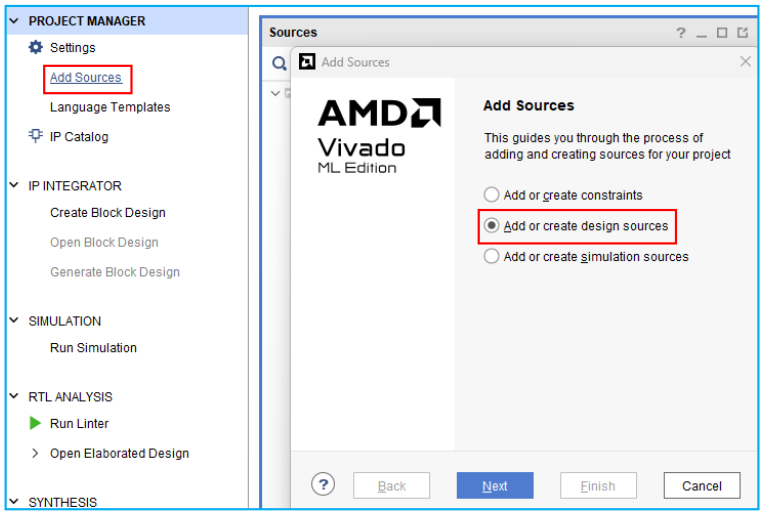

Fig.15. Ventana `Add Sources` para la adición del archivo de jerarquía superior.

19. En la ventana `Add or Create Design Sources` pulsamos en `Add Files` y seleccionamos el archivo correspondiente según el modo seleccionado y pulsamos en `OK` (Fig.16).

20. Antes de pulsar en `Finish`, se debe actualizar el campo `Library` con neorv32 y la opción `Copy sources into project` debe estar seleccionada (Fig.17).

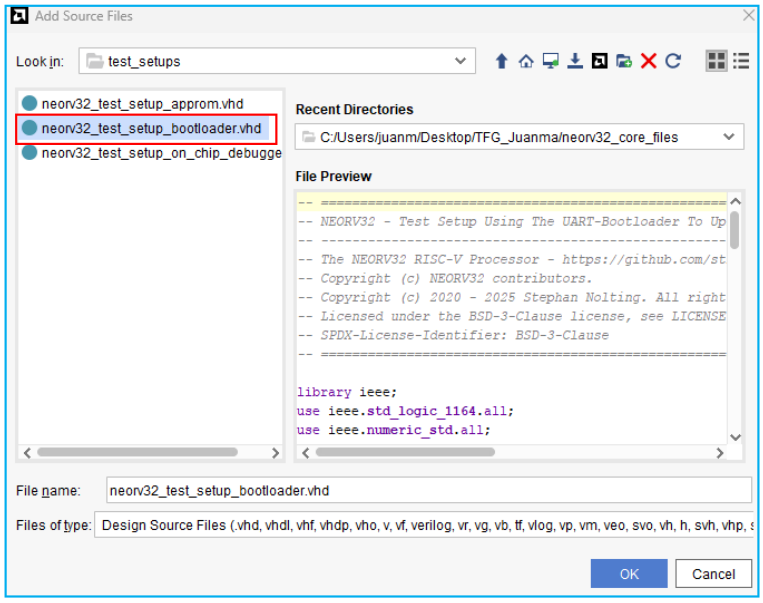

Fig.16. Selección del archivo de jerarquía superior en la ventana `Add Source Files`.

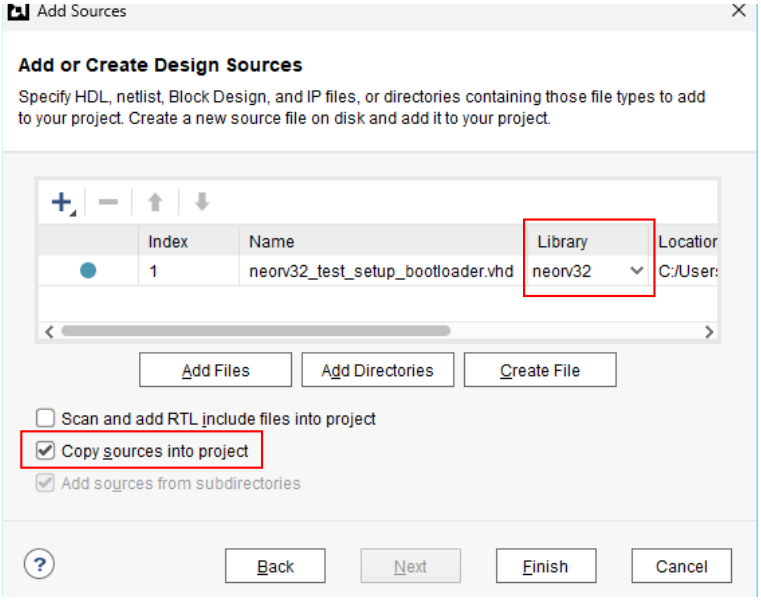

Fig.17. Actualización del campo `Library` y selección de `Copy sources into Project` del archivo de jerarquía superior.

21. Si observamos la jerarquía del proyecto en la pestaña `Hierarchy`, el *test setup* de ha colocado en lo más alto (Fig.18).

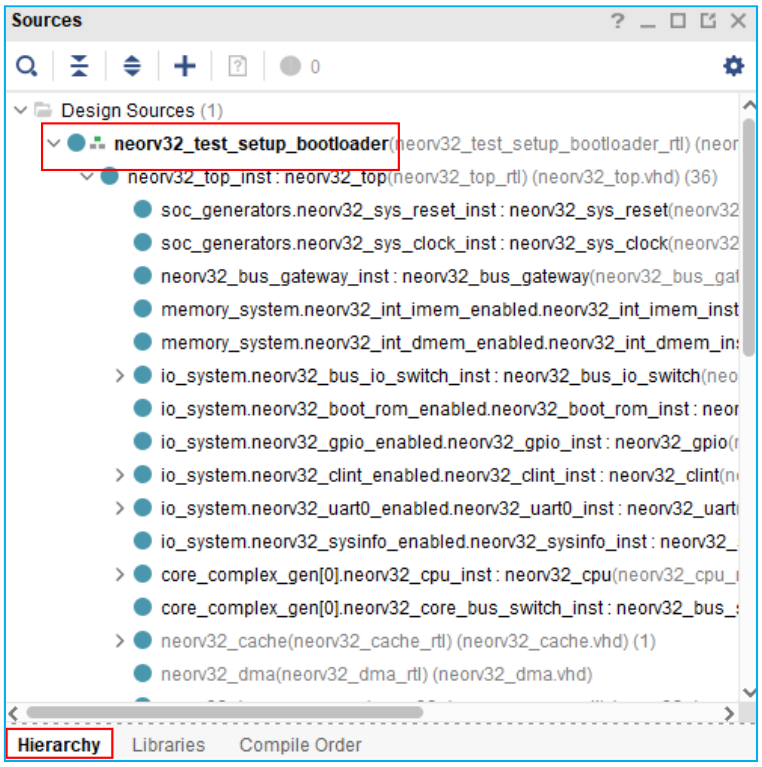

Fig.18. Pestaña `Hierarchy` de la Ventana `Sources`, la entidad *test setup* se coloca en lo más alto de la jerarquía del proyecto.

22. Antes de proseguir con la generación del archivo *bitstream* (`.bit`) del proyecto, se debe modificar el archivo de restricción de pines para configurar el *pinout* de la placa conforme a las necesidades del proyecto. En este caso, se explicará lo mínimo necesario para cubrir con las configuraciones mínimas (*test setups*). En la pestaña `Libraries` dentro de `Sources`, abrir el archivo `Nexys-4-DDR-Master.xdc` alojado en la carpeta `Constraints` (Fig.19).

23. Las modificaciones del archivo de restricción de pines necesarias son las siguientes:

Listado 1. Modificaciones del archivo Nexys-4-DDR-Master.xdc.

```
## Clock signal

set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports {
clk_i }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz

create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
{clk_i}];




##Switches


set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports {
rstn_i }]; #IO_L24N_T3_RS0_15 Sch=sw[0]

#set_property -dict { PACKAGE_PIN L16  IOSTANDARD LVCMOS33 } [get_ports {
SW[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
```

```
# Más contenido por aquí ...


## LEDs


set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]

set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]

set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[2] }]; #IO_L17N_T2_A25_15 Sch=led[2]

set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[3] }]; #IO_L8P_T1_D11_14 Sch=led[3]

set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[4] }]; #IO_L7P_T1_D09_14 Sch=led[4]

set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[5] }]; #IO_L18N_T2_A11_D27_14 Sch=led[5]

set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[6] }]; #IO_L17P_T2_A14_D30_14 Sch=led[6]

set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports {
gpio_o[7] }]; #IO_L18P_T2_A12_D28_14 Sch=led[7]

#set_property -dict { PACKAGE_PIN V16  IOSTANDARD LVCMOS33 } [get_ports {
LED[8] }]; #IO_L16N_T2_A15_D31_14 Sch=led[8]


# Más contenido por aquí ...


##USB-RS232 Interface


set_property -dict { PACKAGE_PIN C4    IOSTANDARD LVCMOS33 } [get_ports {
uart0_rxd_i }]; #IO_L7P_T1_AD6P_35 Sch=uart_txd_in

set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports {
uart0_txd_o }]; #IO_L11N_T1_SRCC_35 Sch=uart_rxd_out

#set_property -dict { PACKAGE_PIN D3   IOSTANDARD LVCMOS33 } [get_ports {
UART_CTS }]; #IO_L12N_T1_MRCC_35 Sch=uart_cts



# Más contenido por aquí ...
```

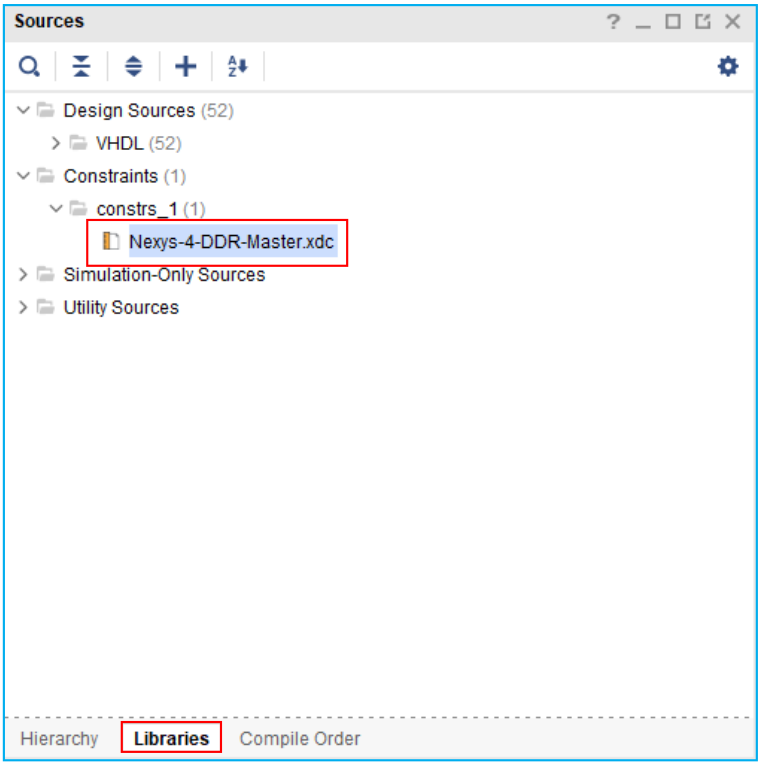

Fig.19. Selección del archivo de restricción de pines para su apertura y posterior modificación.

24. Finalmente, con el archivo de restricción de pines configurado, se puede proseguir con la síntesis, la implementación y la creación del *bitstream*. Para realizar esto se puede pulsar directamente en la opción `Generate Bitstream` (Fig.20).

25. En la ventana dejamos las opciones por defecto y pulsamos `OK`.

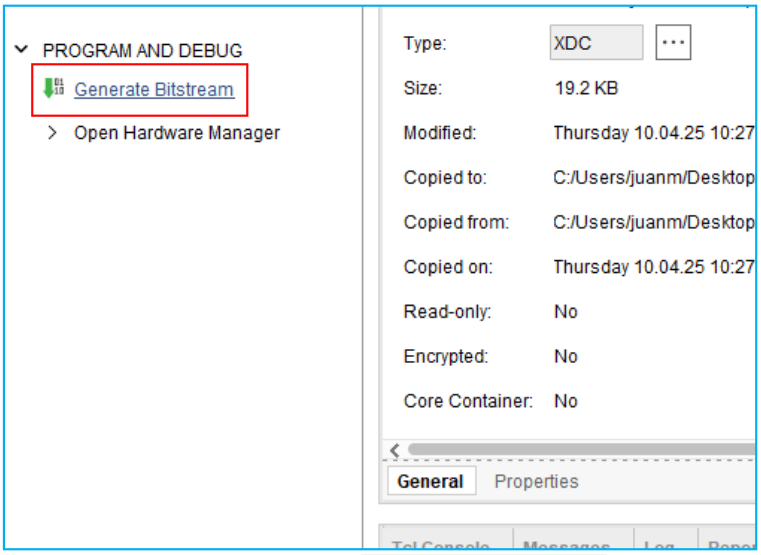

Fig.20. Selección de la opción `Generate Bitstream`, si no se ha realizado la síntesis e implementación del diseño, se realizará automáticamente.

26. A partir de ahora comienza el proceso de síntesis e implementación del diseño, puede tardar varios minutos. Una vez finalizado se indicará del modo que se ilustra en la Fig.21. Además, se abrirá una ventana emergente indicando que se completó la generación del *bitstream*. Si se desea programar la FPGA con el archivo se debe seleccionar la opción `Open Hardware Manager` y pulsar en `OK` (Fig.22).

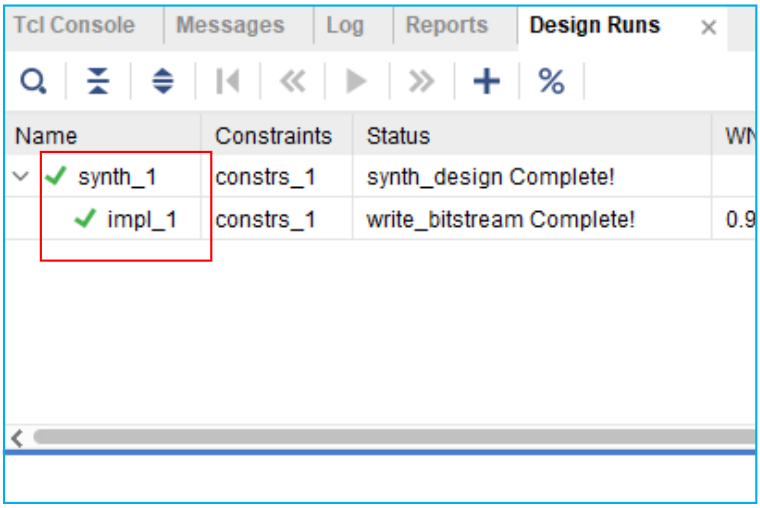

Fig.21. Proceso de síntesis, implementación y generación de *bitstream* exitoso.

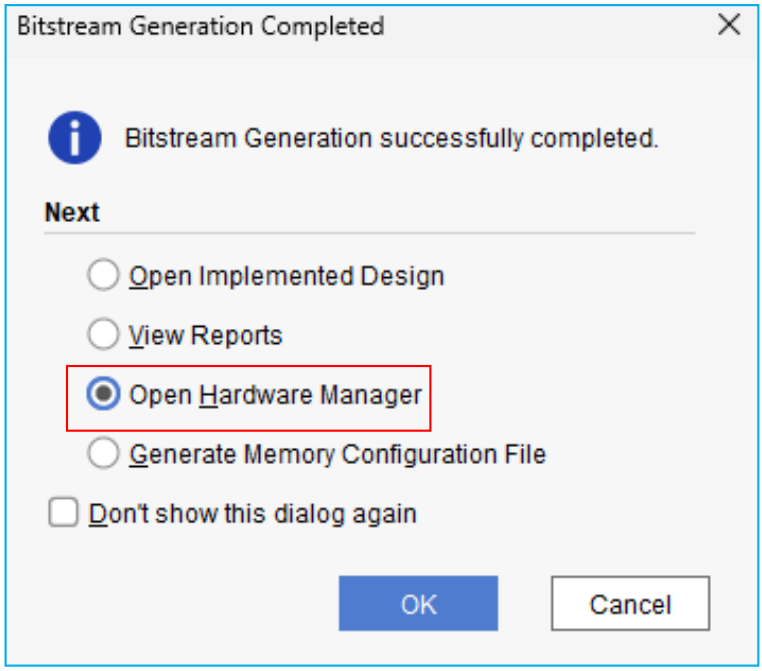

Fig.22. Ventana de finalización de la generación del archivo *bitstream*.

27. Ya en el Hardware Manager, se debe conectar la placa Nexys4 DDR al ordenador. Una vez hecho, seleccionamos la opción `Open Target` y pulsamos en `Autoconnect` (Fig.23).

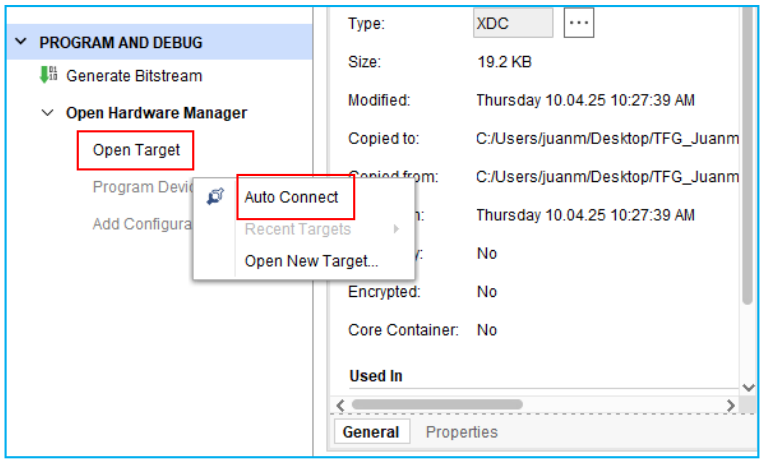

Fig.23. Conexión de la placa Nexys4 DDR a Vivado.

28. Con la placa ya conectada al `Hardware Manger`, seleccionamos la opción `Program Device` y pulsamos en nuestra FPGA `xc7a100t_0` (Fig.24).

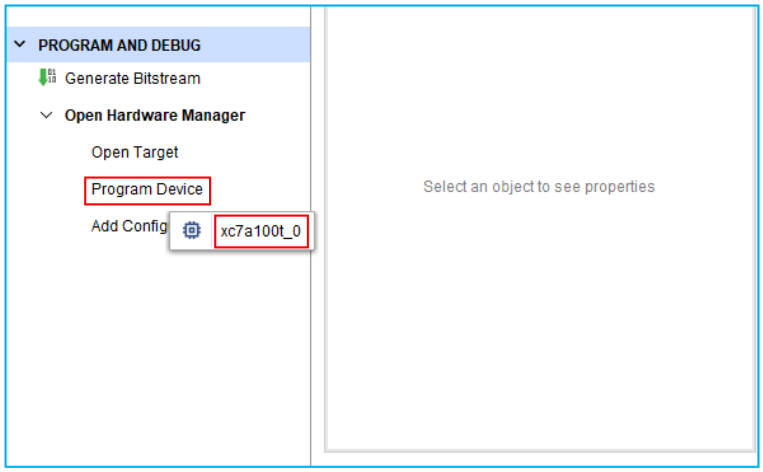

Fig.24. Programación de la FPGA con el archivo *bitstream* generado.

29. Si todo ha ido bien, al activar el *switch* número cero de la placa (`SW0`), el primer LED (`LD0`) debería parpadear (Fig.25.).

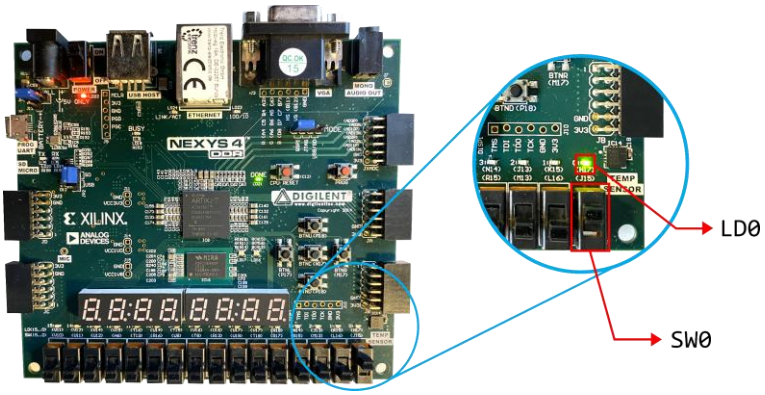

Fig.25. Ilustración de la placa Nexys4 DDR, al cargar el *bitstream*, si se pulsa activa `SW0`, `LD0` comenzará a parpadear.